CS 5402: Introduction to Data Mining

SP2026

(Lecture -3)

Dr. Huiyuan Yang

Department of Computer Science
Missouri S&T

https://hyang428.github.io

MISSOURI S&T

Warm-Up: Why Preprocessing

Which action is **most likely** to improve model performance on real-world data?

A. Switching from logistic regression to a deep neural network

B. Adding more hidden layers

C. Improving data preprocessing and feature representation

D. Increasing training epochs



Learning Objectives

By the end of this lecture, you will be able to:

- Explain why preprocessing is critical in data mining
- Apply normalization and standardization
- Encode categorical features appropriately
- Distinguish feature construction from feature selection
- Build a preprocessing pipeline

Why Preprocessing Matters

Real-world data is:

- Noisy
- Incomplete
- Inconsistent

Most algorithms assume:

- Numerical input
- Comparable feature scales

Preprocessing often improves performance **more than changing models**

Example-1:



You have a dataset of 1,000 student records. The GPA column has 100 missing values (NaN).

	First Name	Gender	Start Date	Last Login Time	Salary	Bonus %	Senior Management	Team
10	Louise	Female	8/12/1980	9:01 AM	63241	15.132	True	NaN
11	Julie	Female	10/26/1997	3:19 PM	102508	12.637	True	Legal
12	Brandon	Male	12/1/1980	1:08 AM	112807	17.492	True	Human Resources
13	Gary	Male	1/27/2008	11:40 PM	109831	5.831	False	Sales
14	Kimberly	Female	1/14/1999	7:13 AM	41426	14.543	True	Finance
15	Lillian	Female	6/5/2016	6:09 AM	59414	1.256	False	Product
16	Jeremy	Male	9/21/2010	5:56 AM	90370	7.369	False	Human Resources
17	Shawn	Male	12/7/1986	7:45 PM	111737	6.414	False	Product
18	Diana	Female	10/23/1981	10:27 AM	132940	19.082	False	Client Services
19	Donna	Female	7/22/2010	3:48 AM	81014	1.894	False	Product
20	Lois	NaN	4/22/1995	7:18 PM	64714	4.934	True	Legal
21	Matthew	Male	9/5/1995	2:12 AM	100612	13.645	False	Marketing
22	Joshua	NaN	3/8/2012	1:58 AM	90816	18.816	True	Client Services
23	NaN	Male	6/14/2012	4:19 PM	125792	5.042	NaN	NaN
24	John	Male	7/1/1992	10:08 PM	97950	13.873	False	Client Services

Operations:

- np.sum()
- np.mean()
- np.std()

```
=== Operations that propagate NaN ===
Sum: nan
Mean: nan
Std Dev: nan
Max: nan
```

Example-2:

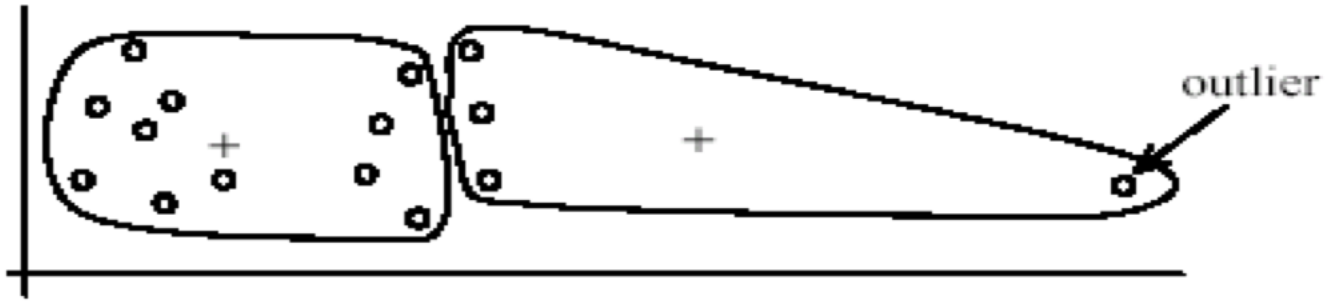


Problem:
We want to predict credit risk (or loan default probability).

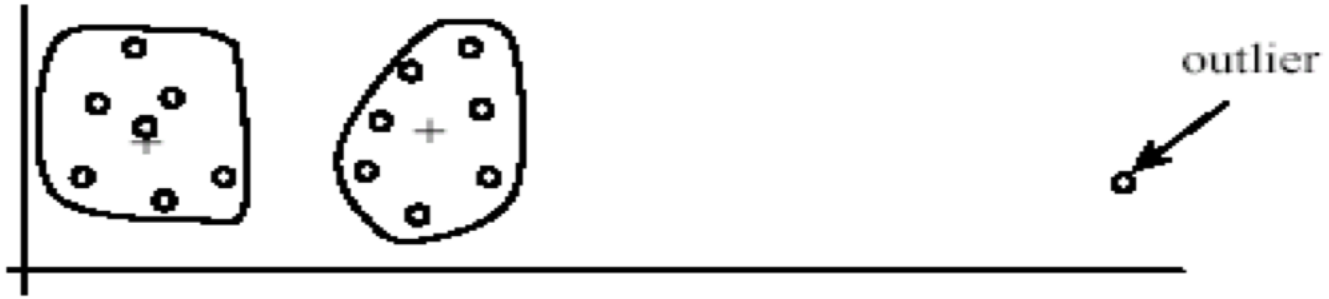
Feature	Meaning	Typical Range
Annual Income	in dollars	20,000 – 200,000
Number of Late Payments	count	0 – 10

$$\text{Distance} \approx (200000 - 50000)^2 + (2 - 1)^2$$

Example-3:

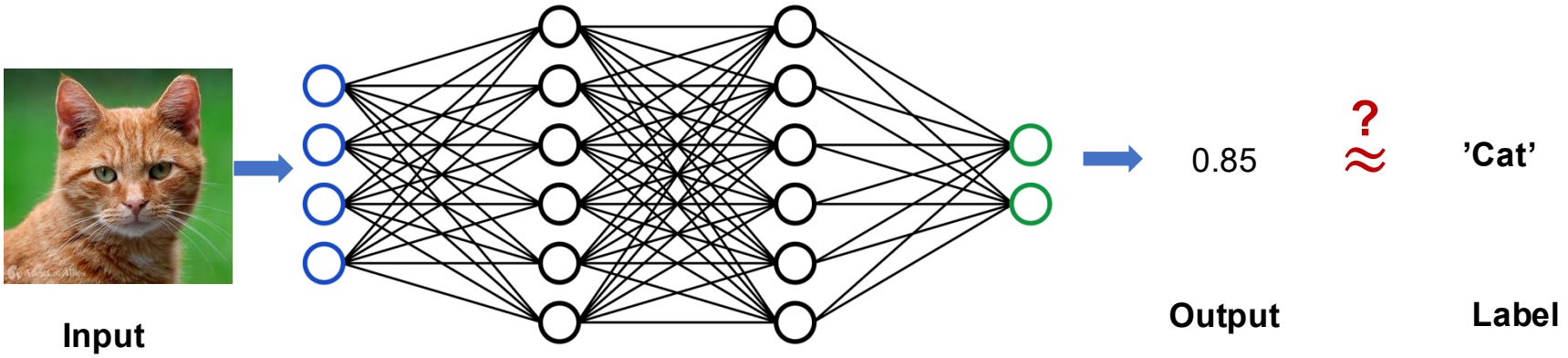


(A): Undesirable clusters



(B): Ideal clusters

Example-4:



Example-5:



Suppose we extract 1,000 features from each email:

Relevant features

- Frequency of words like:
  - "free"
  - "winner"
  - "click"
  - "urgent"
- Number of links
- Presence of ALL-CAPS words

ID	…	Label
1	…	Spam
2	…	Not Spam
3	…	Spam
4	…	Not Spam

Irrelevant / noisy features

- Email ID number
- Time the email was sent (in seconds since epoch)
- Exact message length mod 7
- Random hash values
- HTML tag count that has no relation to spam

These features are easy to collect but not informative.

Outline

- Data Preprocessing
  - Aggregation
  - Missing value
  - Outlier
  - Feature subset selection
  - Sampling
  - Discretization
  - Transformation
- Pipelines

Aggregation

- Combining two or more attributes (or objects) into a single attribute (or object)
- Purpose
  - Data reduction - reduce the number of attributes or objects
  - Change of scale
    - Cities aggregated into regions, states, countries, etc.
    - Days aggregated into weeks, months, or years
  - More "stable" data - aggregated data tends to have less variability

Table 2.4. Data set containing information about customer purchases.

Transaction ID	Item	Store Location	Date	Price	...
⋮	⋮	⋮	⋮	⋮	
101123	Watch	Chicago	09/06/04	$25.99	...
101123	Battery	Chicago	09/06/04	$5.99	...
101124	Shoes	Minneapolis	09/06/04	$75.00	...
⋮	⋮	⋮	⋮	⋮	

Handling Missing Data

There are different strategies for handling missing data:

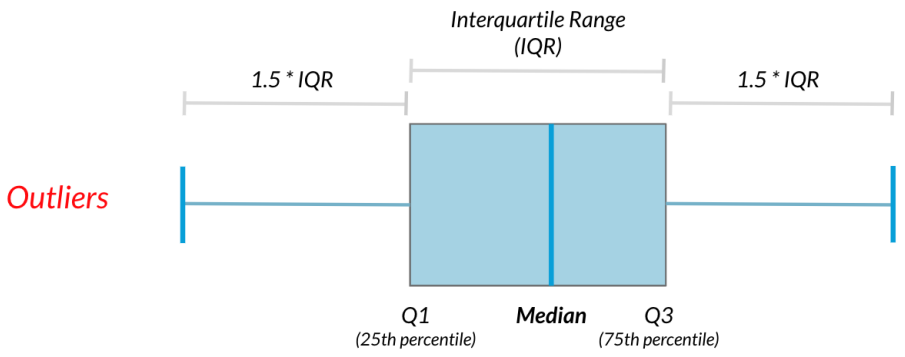
- Remove sample with missing data
  - Simple & Fast, but you lose some data.
- Imputation: Fill in the missing values with a reasonable estimate
  - Mean/Median: for numerical data
  - Mode (most frequent): for categorical data
  - A constant value: filling with 0, ‘unknown’ or another placeholder.

Spotting Outliers

Outlier: A data point that **deviates significantly** from the majority of the data.

Suppose we have employee [ages or salaries($K)]:

[45, 48, 50, 52, 55, 57, 60, 62, 65, 70, 250]



IQR (Boxplot Rule)

- Q1 ≈ 50
- Q3 ≈ 65
- IQR = Q3 − Q1 = 15

Outlier thresholds

- Lower bound: 50 − 1.5 × 15 = 27.5
- Upper bound: 65 + 1.5 × 15 = 87.5

250 is an outlier -> Next??

Normalization (Min-Max Scaling)

This technique scales all data to a fixed range, usually **0 to 1**.

Formula:

$$x_{scaled} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

- The minimum value of the feature becomes 0.
- The maximum value of the feature becomes 1.

Use when:

- Known bounds
- No strong outliers

Pros / Cons

- ✓ Preserves relative order
- ✗ Sensitive to outliers

Standardization (Z-score Scaling)

This technique transforms the data so it has a **mean of 0** and a **standard deviation of 1**.

Formula:

$$x_{scaled} = \frac{x - \mu}{\sigma}$$

Where μ is the mean and σ is the standard deviation.

- The resulting distribution will be centered around 0.
- This method is less affected by outliers than normalization.

Use when:

- Data roughly Gaussian
- Algorithms assume centered data

Common algorithms:

- Linear regression
- Logistic regression
- PCA
- SVM

Feature Construction

Feature construction is the process of **creating new features** from existing data to better represent the underlying problem and improve model performance.

Why Feature Construction is important:

Raw data often:

- Does not directly reflect the true patterns
- Is hard for models to learn from
- Misses meaningful relationships

Feature construction:

- Encodes domain knowledge
- Makes patterns easier to learn
- Can dramatically improve simple models

Feature Construction (Example)

Problem: House Price Prediction (Regression)

Raw features

Feature	Meaning
Total_Price	House price
House_Size	Square feet
Num_Bedrooms	Count

Constructed Feature

Instead of using raw values only, we create:

$$\text{Price_per_SqFt} = \frac{\text{Total_Price}}{\text{House_Size}}$$

Why this helps:

- Normalizes for house size
- Captures market value more directly
- Easier for models to compare houses of different sizes

Feature Selection

- Attribute selection is often important in practice.
  - For example, adding a random (i.e., irrelevant) attribute can significantly degrade performance of a Decision Tree Classifier (C4.5)

Initial dataset

Color	Weight	Texture	Class
Red	150	Smooth	Apple
Orange	200	Rough	Orange
Red	180	Smooth	Apple
Orange	210	Rough	Orange

Adding a random (irrelevant) attribute

Color	Weight	Texture	Random Value	Class
Red	150	Smooth	42	Apple
Orange	200	Rough	84	Orange
Red	180	Smooth	67	Apple
Orange	210	Rough	91	Orange

Feature Selection in Practice

- Attribute selection is often important in practice.
  - For example, adding a random (i.e., irrelevant) attribute can significantly degrade performance of a Decision Tree Classifier (C4.5)
- Feature subset selection targets:
  - Redundant features
    - Duplicate much or all of the information contained in one or more other attributes
    - Example: purchase price of a product and the amount of sales tax paid
  - Irrelevant features
    - Contain no information that is useful for the data mining task at hand
    - Example: students' ID is often irrelevant to the task of predicting students' GPA
- Many techniques developed
- Another way to reduce dimensionality of data

Quick Question-1:

Which action is an example of **feature construction**?

A. Removing highly correlated features

B. Selecting top-k features using mutual information

C. Creating BMI from height and weight

D. Dropping low-variance features



Quick Question-2:

Scenario:

You are predicting **house prices**.

Listing ID	#beds	#baths	#floors	#sqft	Build year	Lot	$price	…
25072557	4	4	2	3,542	2013	0.95 acres	724,900	…
…								

Constructed:

- Price per square foot
- Age of the house

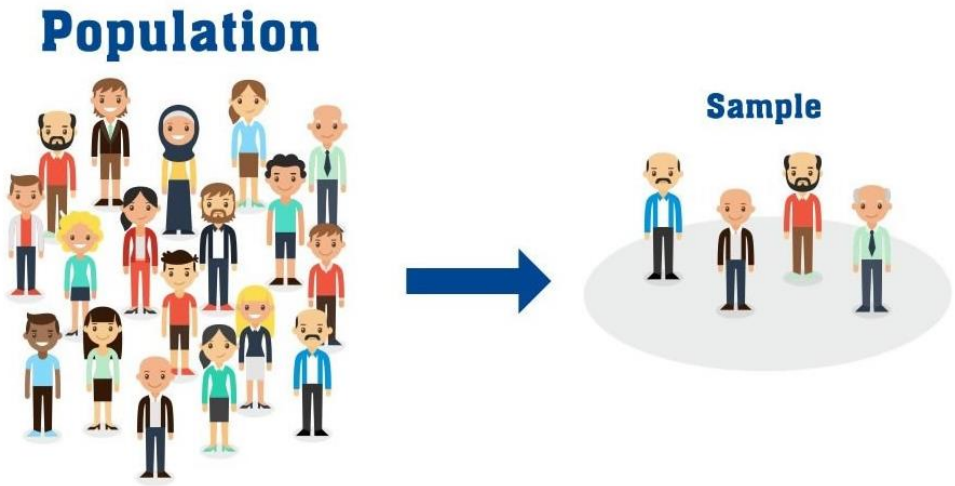
Remove:

- Listing ID



Sampling

- **Sampling** is the process of selecting a subset of data from a larger dataset or population to analyze, study, or make decisions.
  - It is often used for both the preliminary investigation of the data and the final data analysis.
  - It allows researchers, data analysts, and scientists to work with manageable amounts of data while maintaining the ability to make inferences about the entire population.



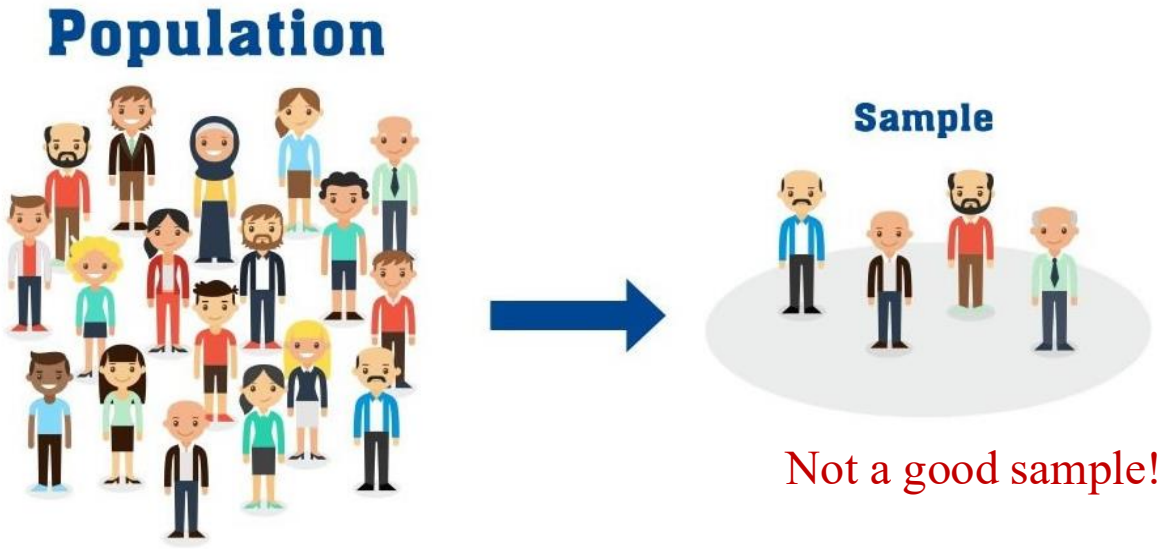
Statisticians often sample because obtaining the entire set of data of interest is too expensive or time consuming.

Sampling: key principle

- Sampling is important
  - *Efficiency*: Analyzing a smaller subset of data is faster and less resource-intensive.
  - *Cost Reduction*: Collecting and processing all the data from a population may be expensive or impractical.
  - *Feasibility*: In many cases, it is impossible to access the entire population (e.g., testing the blood pressure of every individual in a state).
- The key principle for effective sampling:
  - Using a sample will work almost as well as using the entire data set, if the sample is representative
  - A sample is representative if it has approximately the same properties (of interest) as the original set of data

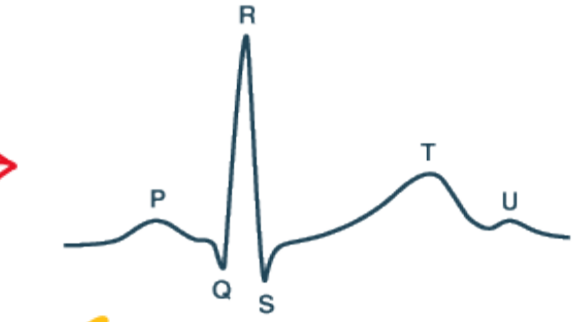
Sampling: Challenges

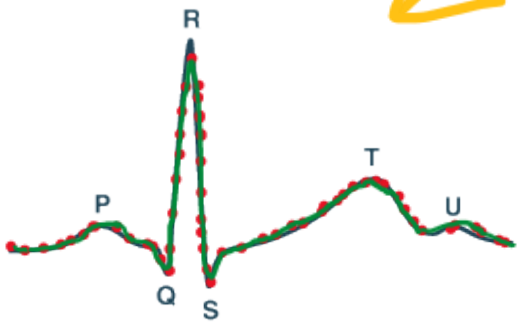
- Challenges in Sampling:
  - *Bias*: Poor sampling methods can lead to unrepresentative samples, introducing bias.
  - *Sample Size*: Too small a sample may not represent the population accurately, while a very large sample might be unnecessary.
  - *Randomness*: Ensuring that selection is truly random can sometimes be difficult.



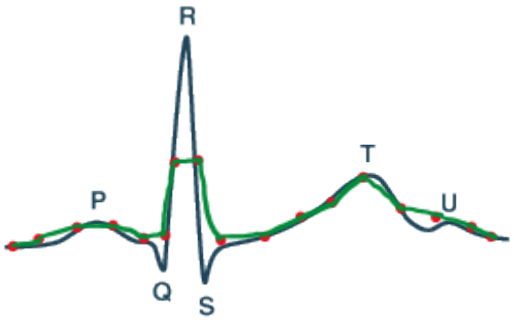
Example: Sample Size







[128, 1]



[16, 1]

Types of Sampling

- Simple Random Sampling
  - There is an equal probability of selecting any particular item
  - Sampling without replacement
    - As each item is selected, it is removed from the population
  - Sampling with replacement
    - Objects are not removed from the population as they are selected for the sample.
    - In sampling with replacement, the same object can be picked up more than once
- Stratified sampling
  - Split the data into several partitions; then draw random samples from each partition



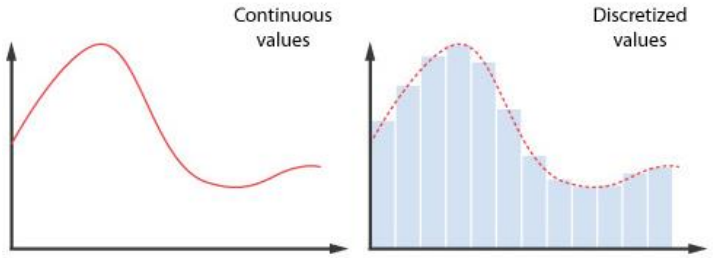


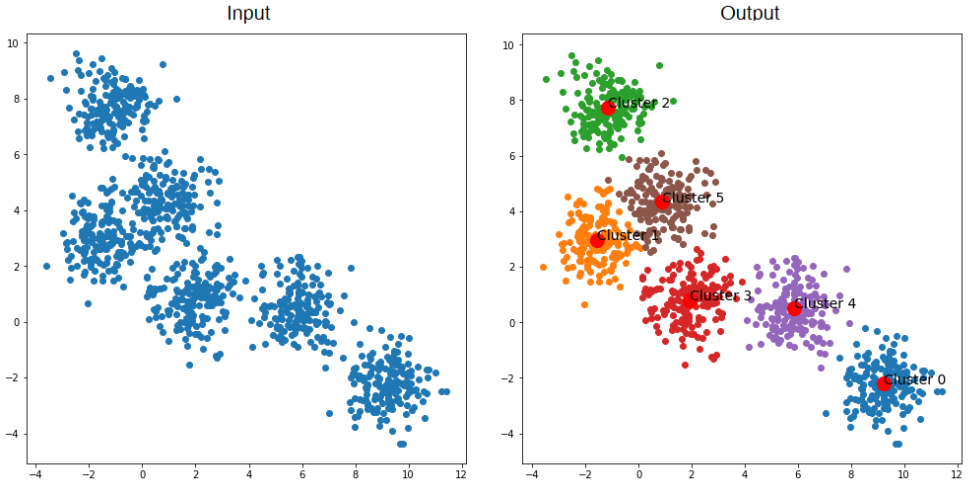
Discretization

- Discretization is the process of converting a continuous attribute into discrete categories or intervals.
- This process is usually the first step toward making the attribute suitable for numerical analysis and implementation.
  - Particularly for algorithms that work better with categorical data or for tasks where interpretability is important.
- Benefits of discretization, includes:
  i. *Simplifies Data*: Reduces complexity by dividing continuous ranges into manageable segments. Prepares data for further analysis. Many DM algorithms which can only deal with discrete attributes
  ii. *Improve Model Performance and Efficiency*: Some algorithms, like decision trees or rule-based classifiers, perform better with categorical inputs. Reduce the running time of various data mining tasks;
  iii. *Enables Interpretability*: Helps understanding and interpreting the relationships between attributes.

Discretization Examples

- Typical methods for data Discretization
  - Equal-Width Binning
  - Equal-Frequency Binning
  - Clustering-based Discretization
  - Entropy-Based Discretization





Define age groups (bins):

- Young: 18–29
- Adult: 30–49
- Senior: 50–70

Transform numeric feature → categorical feature:

From Categories to Numbers

Machine learning models are math-based, which means they need numbers, not text.

Example: $Loss = 0.85 - 'cat'$ **ERROR!**

We must convert categorical variables like major or city into a numerical format. This is called **encoding**.

Two main methods:

- Label Encoding
- One-Hot Encoding

Label Encoding: Categories to Integers

Label Encoding assigns a unique integer to each category.

Major	Major_Encoded
CS	0
Math	1
Physics	2
CS	0

Warning: This creates an artificial order (CS < Math < Physics). This can be a problem for some models.

Use Case: Only appropriate for **ordinal** variables where the rank matters (e.g., 'Small'=0, 'Medium'=1, 'Large'=2).

One-Hot Encoding: The Gold Standard

One-Hot Encoding creates new binary (0 or 1) columns for each category.

Major
CS
Math
Physics
CS

Major_CS	Major_Math	Major_Physics
1	0	0
0	1	0
0	0	1
1	0	0

Pros:
No artificial order is created. This is the **safest and most common** approach for nominal data.

Quick Question-3:

Which categorical feature should **NOT** be label-encoded?

A. Education level (High School, Bachelor, Master)

B. Product category (Laptop, Phone, Tablet)

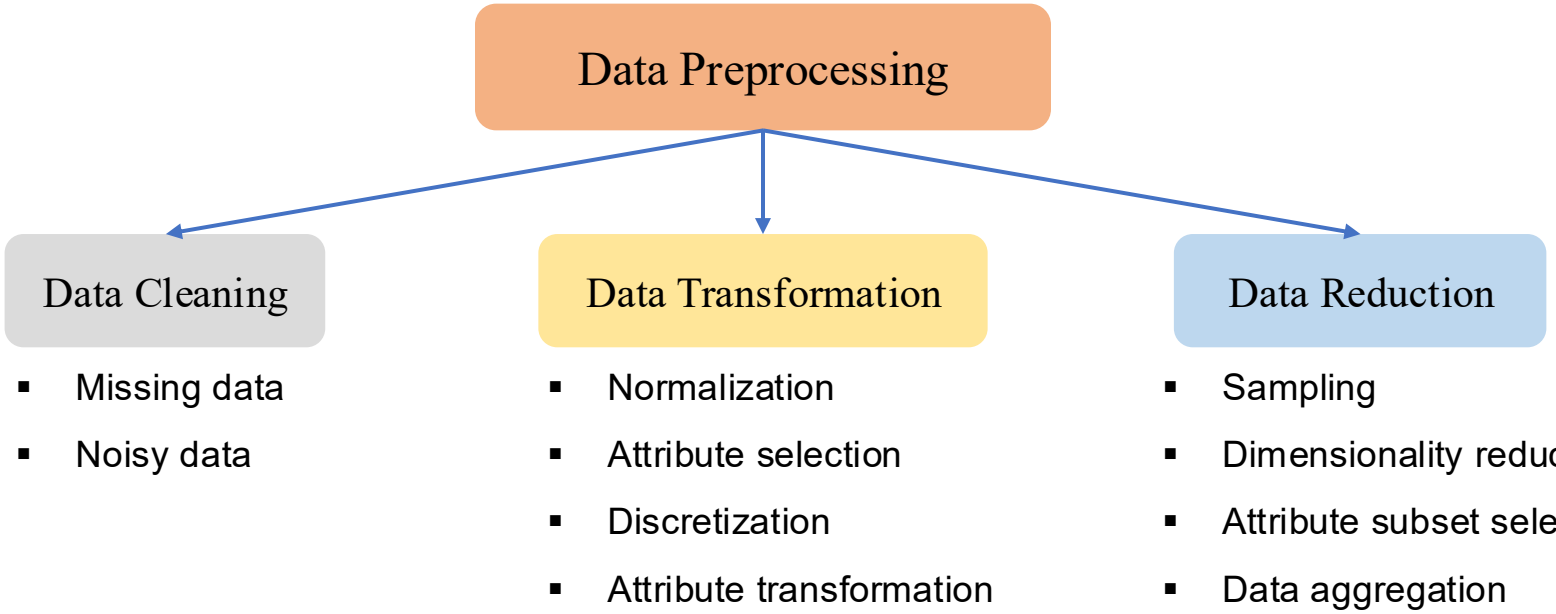
C. Satisfaction level (Low, Medium, High)

D. Risk level (Low, Medium, High)



Summary of Data Preprocessing

- Data preprocessing is an important step in the data mining process. It refers to the cleaning, transforming, and integrating of data in order to make it ready for analysis.
- The goal of data preprocessing is to improve the quality of the data and to make it more suitable for the specific data mining task.
- Some common steps in data preprocessing include:



What is a Data Preprocessing Pipeline

A **data preprocessing pipeline** is an **ordered sequence of steps** that transforms raw data into model-ready data **consistently and without data leakage**.

Why Pipelines Matter:

- Raw data is messy, inconsistent, and incomplete
- Models assume clean, numeric, well-scaled input
- Pipelines ensure:
  - Correct order
  - Reproducibility
  - No data leakage (Fair evaluation)
  - Correct deployment

Common Pipeline Mistakes (Summary)

Mistake	Core Issue
Normalize before split	Data leakage
Feature selection on full data	Overfitting
Blind outlier removal	Bias
Wrong categorical encoding	Fake order
No scaling for distance models	Dominance
Preprocess outside CV	Leakage
Drop missing data blindly	Bias
Mismatched pipelines	Deployment failure

Common Data Preprocessing Pipeline

1) Raw Data.
2) Train / Test Split
3) Missing Value Handling
4) Outlier Handling (if needed)
5) Feature Construction
6) Categorical Encoding
7) Feature Scaling
8) Feature Selection (optional)
9) Model Training

Golden Rules

- Split data before preprocessing
- Fit preprocessing only on training data
- Apply the same steps to test and future data
- Use pipelines to avoid leakage

Summary

- Preprocessing is **not optional**
- Scaling matters for many algorithms
- Encoding must respect semantics
- Feature engineering balances signal and complexity
- Pipelines enforce best practices

Next Lecture

- **Similarity, Distance & Evaluation Basics**
  - Distance metrics（L1, L2, cosine, Jaccard）
  - Similarity for different data types
  - Train/validation/test
  - Confusion matrix, accuracy, precision, recall, F1

Practice after lecture (1)

- Create a toy dataset in Python (Pandas) with Quality issues:

```
import pandas as pd
import numpy as np

def create_dirty_dataset():
    data = {
        'CustomerID': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
        'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve', 'Frank', 'George', 'Hannah', 'Ian', 'Jack'],
        'Age': [25, 32, np.nan, 45, 28, 32, 51, 23, 34, 29],
        'Gender': ['Female', 'Male', 'Male', 'Male', 'Female', 'Male', 'male', 'Female', 'Male', np.nan],
        'PurchaseAmount': [150.50, 80.00, 200.00, -50.00, 120.00, 80.00, 9999.99, 45.00, 150.50, 75.00],
        'PurchaseDate': ['2025-01-10', '2025-01-12', '2025-01-15', '2025-01-01', '2025/01/20', 'invalid-date', '2025-02-01', '2025-02-
05', '2025-02-08', '2025-02-10']
    }
    df = pd.DataFrame(data)

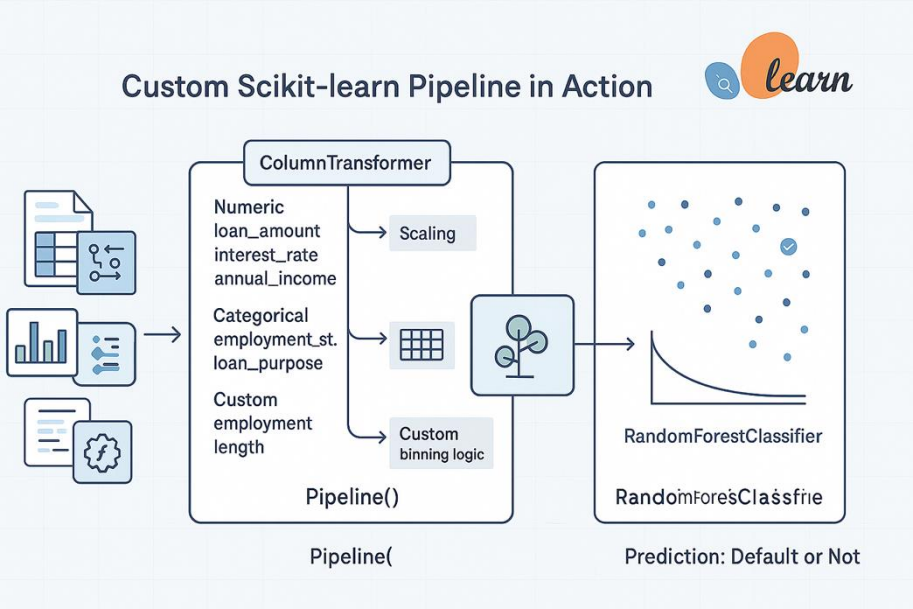
    # Introduce specific quality issues:
    # 1. Missing values (Age and Gender)
    # 2. Inconsistent formatting (Gender: 'male' vs 'Male'; Date format: 'YYYY-MM-DD' vs 'YYYY/MM/DD'; Date: 'invalid-date')
    # 3. Outliers (PurchaseAmount: -50.00 and 9999.99)
    # 4. Duplicate record (CustomerID 2 and 6 have same PurchaseAmount and Age) - correction: rows 2 and 6 are different IDs
but same age/amount, so not a full duplicate record, but good for analysis
    # 5. Duplicates (CustomerID 1 and 9 have the exact same PurchaseAmount, good for checking transactional duplicates)

    return df

# To use the dataset:
toy_data_df = create_dirty_dataset()
print(toy_data_df)
```

Practice after lecture (2)

- Build a data preprocessing pipeline




```
numeric_preprocessor = Pipeline(
    steps=[
        ("imputation_mean", SimpleImputer(missing_values=np.nan, strategy="me
        ("scaler", StandardScaler()),
    ]
)

categorical_preprocessor = Pipeline(
    steps=[
        (
            "imputation_constant",
            SimpleImputer(fill_value="missing", strategy="constant"),
        ),
        ("onehot", OneHotEncoder(handle_unknown="ignore")),
    ]
)

preprocessor = ColumnTransformer(
    [
        ("categorical", categorical_preprocessor, ["state", "gender"]),
        ("numerical", numeric_preprocessor, ["age", "weight"]),
    ]
)
```